Politechnika Śląska

UCZELNIA BADAWCZA
INICJATYWA DOSKONAŁOŚCI

Sprawozdanie

Rodzaj zajęć
Laboratorium

Przedmiot
Systemy Mikroprocesorowe i Wbudowane

Rok akademicki	Miasto	Kierunek	Semestr	Prowadzący	Grupa	Sekcja
2025/2026	G	INF	5	JP	5	10

Planowany termin wykonywania ćwiczenia		Faktyczny termin wykonania ćwiczenia	
Data	Godzina	Data	Godzina
12.01.2026	09:45		

Numer ćwiczenia	Temat ćwiczenia
23	Mikrokontrolery AVR cz.3. – Liczniki i Przerwania

Skład sekcji		
	Imię i nazwisko	Uwagi
1	Krzysztof Czerwonka	

1. Treść zadania

Proszę wyłączyć wszystkie diody na początku programu. Poczekaj na naciśnięcie przycisku. Kopiowanie tabel rozpoczyna się po naciśnięciu dowolnego przycisku na porcie A. Napisz program, który kopiuje każdy bajt mniejszy niż 0x80 z pamięci TAB_ROM do pamięci TAB_RAM, po jednym bajcie na raz. Jeśli pamięć TAB_RAM jest większa niż pamięć TAB_ROM, kopiowanie jest kontynuowane od początku pamięci TAB_ROM po osiągnięciu końca zawijając się, aby całkowicie wypełnić pamięć TAB_RAM. Wyświetl każdy skopiowany bajt na diodach podłączonych do portu C. Odstęp pomiędzy wyświetlaniem każdego bajtu powinien wynosić około 500 ms 1000 ms. Po zakończeniu programu włącz diodę LED podłączoną do portu B.7. Następnie: Włącz diodę LED podłączoną do portu B.7 dokładnie: na 1000 ms - włącz, następne 250 ms - wyłącz, następne na 1000 ms - włącz, a następnie wyłącz.

2. Kod programu

```
////////////////////////////////////////////////////////////////////////////
// Laboratory AVR Microcontrollers Part2
// Program template for lab 9
// Please fill in this information before starting coding
// Authors: Krzysztof Czerwonka
//
// Group: 5
// Section: 10
//
// Task: 7
//
//
//
// Version: 5.0
////////////////////////////////////////////////////////////////////////////
.nolist ;quartz assumption 4Mhz
.include "m2560def.inc"
;////////////////////////////////////////////////////////////////////////////
.list
.equ xlength =      100
;////////////////////////////////////////////////////////////////////////////
; EEPROM - data non volatile memory segment
.ESEG

;////////////////////////////////////////////////////////////////////////////
; StaticRAM - data memory.segment
.DSEG
.ORG 0x200; may be omitted this is default value
; Destination table (xlengthx bytes).
; Replace "xlengthx" with correct value
TAB_RAM: .BYTE xlength

;////////////////////////////////////////////////////////////////////////////
; CODE - Program memory segment
; Please Remember that it is "word" address space
;
.CSEG


; Interrupts vector table / change to your procedure only when needed
```

```
jmp    EXT_INT0      ; IRQ0 Handler
jmp    EXT_INT1      ; IRQ1 Handler
jmp    EXT_INT2      ; IRQ2 Handler
jmp    EXT_INT3      ; IRQ3 Handler
jmp    EXT_INT4      ; IRQ4 Handler
jmp    EXT_INT5      ; IRQ5 Handler
jmp    EXT_INT6      ; IRQ6 Handler
jmp    EXT_INT7      ; IRQ7 Handler
jmp HPCINT0          ; PCINT0 Handler
jmp HPCINT1          ; PCINT1 Handler
jmp HPCINT2          ; PCINT2 Handler
jmp WDT                    ; WDT Handler
jmp    TIM2_COMPA    ; Timer2 CompareA Handler
jmp    TIM2_COMPB    ; Timer2 CompareB Handler
jmp    TIM2_OVF      ; Timer2 Overflow Handler
jmp    TIM1_CAPT     ; Timer1 Capture Handler
jmp    TIM1_C0MPA    ; Timer1 CompareA Handler
jmp    TIM1_C0MPB    ; Timer1 CompareB Handler
jmp TIM1_COMPC       ; Timer1 CompareC Handler
jmp    TIM1_0VF      ; Timer1 Overflow Handler
jmp    TIM0_COMPA    ; Timer0 CompareA Handler
jmp    TIM0_COMPB    ; Timer0 CompareB Handler
jmp    TIM0_OVF      ; Timer0 Overflow Handler
jmp    SPI_STC       ; SPI Transfer Complete Handler
jmp    USART0_RXC    ; USART0 RX Complete Handler
jmp    USART0_UDRE   ; USART0,UDR Empty Handler
jmp    USART0_TXC    ; USART0 TX Complete Handler
jmp ANA_COMP  ; Analog COmparator Handler
jmp    HADC          ; ADC Conversion Complete Handler
jmp    EE_RDY        ; EEPROM Ready Handler
jmp    TIM3_CAPT     ; Timer3 Capture Handler
jmp    TIM3_COMPA    ; Timer3 CompareA Handler
jmp    TIM3_COMPB    ; Timer3 CompareB Handler
jmp    TIM3_COMPC    ; Timer3 CompareC Handler
jmp    TIM3_OVF      ; Timer3 Overflow Handler
jmp    USART1_RXC    ; USART1 RX Complete Handler
jmp    USART1_UDRE   ; USART1,UDR Empty Handler
jmp    USART1_TXC    ; USART1 TX Complete Handler
jmp    TWI           ; Two-wire Serial Interface Interrupt Handler
jmp    SPM_RDY       ; SPM Ready Handler
jmp TIM4_CAPT ; Timer4 Capture Handler
jmp TIM4_COMPA       ; Timer4 CompareA Handler
jmp TIM4_COMPB       ; Timer4 CompareB Handler
jmp TIM4_COMPC       ; Timer4 CompareC Handler
jmp TIM4_OVF  ; Timer4 Overlflow Handler
jmp TIM5_CAPT ; Timer5 Capture Handler
jmp TIM5_COMPA       ; Timer5 CompareA Handler
jmp TIM5_COMPB       ; Timer5 CompareB Handler
jmp TIM5_COMPC       ; Timer5 CompareC Handler
jmp TIM5_OVF  ; Timer5 Overlflow Handler
jmp    USART2_RXC    ; USART2 RX Complete Handler
jmp    USART2_UDRE   ; USART2,UDR Empty Handler
jmp    USART2_TXC    ; USART2 TX Complete Handler
jmp    USART3_RXC    ; USART3 RX Complete Handler
jmp    USART3_UDRE   ; USART3,UDR Empty Handler
jmp    USART3_TXC    ; USART3 TX Complete Handler

////////////////////////////////////////////////////////////////////////////////
EXT_INT0:     ; IRQ0 Handler
EXT_INT1:     ; IRQ1 Handler
EXT_INT2:     ; IRQ2 Handler
EXT_INT3:     ; IRQ3 Handler
```

```
EXT_INT4:     ; IRQ4 Handler
EXT_INT5:     ; IRQ5 Handler
EXT_INT6:     ; IRQ6 Handler
EXT_INT7:     ; IRQ7 Handler
HPCINT0:            ; PCINT0 Handler
HPCINT1:            ; PCINT1 Handler
HPCINT2:            ; PCINT2 Handler
WDT:          ; WDT Handler
TIM2_COMPA:   ; Timer2 CompareA Handler
TIM2_COMPB:   ; Timer2 CompareB Handler
TIM2_OVF:     ; Timer2 Overflow Handler
TIM1_CAPT:    ; Timer1 Capture Handler
TIM1_C0MPA:   ; Timer1 CompareA Handler
TIM1_C0MPB:   ; Timer1 CompareB Handler
TIM1_COMPC:   ; Timer1 CompareC Handler
TIM1_0VF:     ; Timer1 Overflow Handler
TIM0_COMPA: ; Timer0 CompareA Handler
TIM0_COMPB: ; Timer0 CompareB Handler
TIM0_OVF:     ; Timer0 Overflow Handler
SPI_STC:      ; SPI Transfer Complete Handler
USART0_RXC:   ; USART0 RX Complete Handler
USART0_UDRE:; USART0,UDR Empty Handler
USART0_TXC:   ; USART0 TX Complete Handler
ANA_COMP:     ; Analog COmparator Handler
HADC:       ; ADC Conversion Complete Handler
EE_RDY:       ; EEPROM Ready Handler
TIM3_CAPT:    ; Timer3 Capture Handler
TIM3_COMPA:   ; Timer3 CompareA Handler
TIM3_COMPB:   ; Timer3 CompareB Handler
TIM3_COMPC:   ; Timer3 CompareC Handler
TIM3_OVF:     ; Timer3 Overflow Handler
USART1_RXC:   ; USART1 RX Complete Handler
USART1_UDRE:; USART1,UDR Empty Handler
USART1_TXC:   ; USART1 TX Complete Handler
TWI:          ; Two-wire Serial Interface Interrupt Handler
SPM_RDY:      ; SPM Ready Handler
TIM4_CAPT:    ; Timer4 Capture Handler
TIM4_COMPA:   ; Timer4 CompareA Handler
TIM4_COMPB:   ; Timer4 CompareB Handler
TIM4_COMPC:   ; Timer4 CompareC Handler
TIM4_OVF:     ; Timer4 Overlflow Handler
TIM5_CAPT:    ; Timer5 Capture Handler
TIM5_COMPA:   ; Timer5 CompareA Handler
TIM5_COMPB:   ; Timer5 CompareB Handler
TIM5_COMPC:   ; Timer5 CompareC Handler
TIM5_OVF:     ; Timer5 Overlflow Handler
USART2_RXC:   ; USART2 RX Complete Handler
USART2_UDRE:; USART2,UDR Empty Handler
USART2_TXC:   ; USART2 TX Complete Handler
USART3_RXC:   ; USART3 RX Complete Handler
USART3_UDRE:; USART3,UDR Empty Handler
USART3_TXC:   ; USART3 TX Complete Handler
       reti          ; return from all no used

;////////////////////////////////////////////////////////////////////////////
.org 0x0100
RESET:
    cli
    ldi R16, HIGH(RAMEND)
    out SPH, R16
    ldi R16, LOW(RAMEND)
    out SPL, R16
```

```
    ; --- PUNKT 4: INICJALIZACJA WSZYSTKICH PORTÓW A-L ---

    ; PORT A: Przyciski (Schemat Sheet 3/3). Wejście z Pull-up.
    clr R16
    out DDRA, R16
    ser R16
    out PORTA, R16

    ; PORT B: PB5, PB6, PB7 jako wyjścia (Schemat Sheet 2/3).
    ldi R16, (1<<DDB5)|(1<<DDB6)|(1<<DDB7)
    out DDRB, R16
    clr R16
    out PORTB, R16 ; Startowo OFF (Logika dodatnia)

    ; PORT C: Wyjścia LED (Schemat Sheet 2/3).
    ; Zworka SV1 na L = Logika dodatnia (H zapala diodę).
    ser R16
    out DDRC, R16
    clr R16
    out PORTC, R16 ; Startowo OFF

    ; PORT E: PE4 jako wyjście (Dioda US1 na schemacie Sheet 2/3).
    sbi DDRE, 4
    cbi PORTE, 4 ; OFF

    ; Inicjalizacja pozostałych portów jako wejścia bez pull-up (Punkt 4)
    clr R16
    out DDRD, R16
    out PORTD, R16
    sts DDRF, R16
    sts PORTF, R16
    sts DDRG, R16
    sts PORTG, R16
    sts DDRH, R16
    sts PORTH, R16
    sts DDRJ, R16
    sts PORTJ, R16
    sts DDRK, R16
    sts PORTK, R16
    sts DDRL, R16
    sts PORTL, R16

    ; Konfiguracja Timera 1 (Preskaler 256 dla 16MHz)
    ldi R16, (1<<CS12)
    sts TCCR1B, R16

; --- Oczekiwanie na przycisk (L = naciśnięty) ---
Wait_For_Button:
    in R16, PINA
    com R16           ; Inwersja, aby naciśnięty przycisk (L) był widziany jako 1
    andi R16, 0xFF
    breq Wait_For_Button
    rcall DELAY_20MS

; --- PUNKT 3: KOPIOWANIE BEZ LPM, SBIW, ADIW ---
    ldi XH, high(TAB_RAM)
    ldi XL, low(TAB_RAM)
    ldi R24, xlength

Reload_ROM:
    ldi ZH, high(TAB_ROM<<1)
```

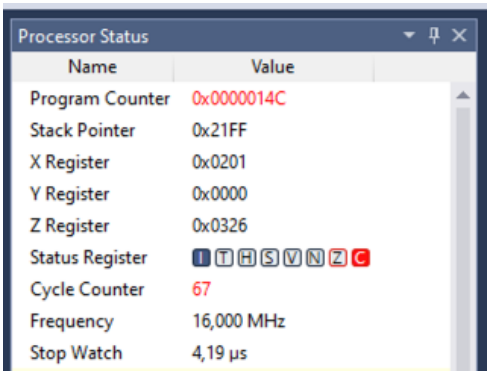
```
    ldi ZL, low(TAB_ROM<<1)
    ldi R16, byte3(TAB_ROM<<1)
    out RAMPZ, R16

Copy_Loop:
    elpm R17, Z+        ; Używamy ELPM (Zgodne z ATmega2560)

    ; Sprawdzanie terminatora (0,0)
    tst R17
    brne Filter_Check
    elpm R18, Z
    tst R18
    breq Reload_ROM     ; Zapętlenie ROM

Filter_Check:
    ; Kopiuj tylko bajty < 0x80
    cpi R17, 0x80
    brsh Skip_Byte_Label

    st X+, R17

    ; --- PUNKT 6: POMIARY I WYŚWIETLANIE (LOGIKA DODATNIA) ---
    ; (1) Zrzut: Przed zapaleniem
    out PORTC, R17      ; Zapal LEDy (H = ON zgodnie ze zworką na L)

    rcall WAIT_1S

    ; (2) Zrzut: Po zgaszeniu
    clr R16
    out PORTC, R16      ; Zgaś LEDy

    subi R24, 1
    breq Final_Sequence_Jump
    jmp Copy_Loop       ; Naprawa błędu zasięgu

Skip_Byte_Label:
    jmp Copy_Loop

Final_Sequence_Jump:
    jmp Final_Sequence

Final_Sequence:
    clr R16
    out PORTC, R16

    ; (3) Zrzut: PB7 ON
    sbi PORTB, 7
    rcall WAIT_1S

    ; (4) Zrzut: PB7 OFF (Czas świecenia 1: 1s)
    cbi PORTB, 7
    rcall WAIT_250MS

    ; (5) Zrzut: PB7 ON ponownie (Czas przerwy: 250ms)
    sbi PORTB, 7
    rcall WAIT_1S

    ; (6) Zrzut: PB7 OFF (Czas świecenia 2: 1s)
    cbi PORTB, 7

//-------------------------------------------------------------------------------
// Program end - Ending loop
```

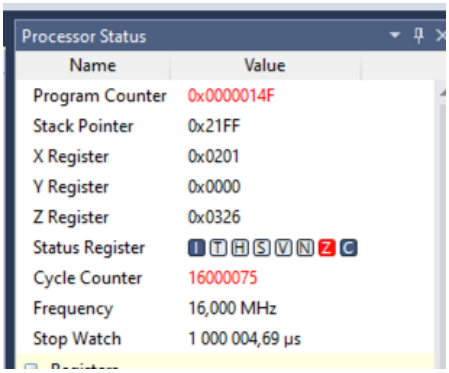
```
//-----------------------------------------------------------------------------
End:
       rjmp END

; ==========================================================
; PROCEDURY CZASOWE
; ==========================================================

WAIT_250MS:
    push R16
    ldi R16, 0xC2
    sts TCNT1H, R16
    ldi R16, 0xF7
    sts TCNT1L, R16
    rjmp TIMER_WAIT

WAIT_1S:
    push R16
    ldi R16, 0x0B
    sts TCNT1H, R16
    ldi R16, 0xDC
    sts TCNT1L, R16
    rjmp TIMER_WAIT

TIMER_WAIT:
    ldi R16, (1<<TOV1)
    out TIFR1, R16      ; Czyść flagę przepełnienia
Wait_L:
    in R16, TIFR1
    sbrs R16, TOV1
    rjmp Wait_L
    pop R16
    ret

DELAY_20MS:
    push R19
    push R21
    push R22
    ldi R19, 4
L_O: ldi R21, 105
L_M: ldi R22, 255
L_I: dec R22
    brne L_I
    dec R21
    brne L_M
    dec R19
    brne L_O
    pop R22
    pop R21
    pop R19
    ret

//-----------------------------------------------------------------------------
// Table Declaration -  place here test values
// Test with different table values and different begin addresses of table (also above
0x8000)
//
//.org 0x8000
TAB_ROM:      .db    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A,
0x0B, 0x0C, 0x0D, 0x0E, 0x1F
                     .db          0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1F, 0x1F
```

```
                    .db          0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27,
0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F
                    .db          0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37,
0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F
                    .db          0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47,
0x48, 0x49, 0x4A, 0x4B, 0x4C, 0x4D, 0x4E, 0x4F
                    .db          0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57,
0x58, 0x59, 0x5A, 0x5B, 0x5C, 0x5D, 0x5E, 0x5F
                    .db          0x60, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67,
0x68, 0x69, 0x6A, 0x6B, 0x6C, 0x6D, 0x6E, 0x6F
                    .db          0x70, 0x71, 0x72, 0x73, 0x74, 0x75, 0x76, 0x77,
0x78, 0x79, 0x7A, 0x7B, 0x7C, 0x7D, 0x7E, 0x7F
                    .db          0x80, 0x81, 0x82, 0x83, 0x84, 0x85, 0x86, 0x87,
0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8D, 0x8E, 0x8F
                    .db          0x90, 0x91, 0x92, 0x93, 0x94, 0x95, 0x96, 0x97,
0x98, 0x99, 0x9A, 0x9B, 0x9C, 0x9D, 0x9E, 0x9F
                    .db          0xa0, 0xa1, 0xa2, 0xa3, 0xa4, 0xa5, 0xa6, 0xa7,
0xa8, 0xa9, 0xaA, 0xaB, 0xaC, 0xaD, 0xAE, 0xAF
                    .db          0xB0, 0xB1, 0xB2, 0xB3, 0xB4, 0xB5, 0xB6, 0xB7,
0xB8, 0xB9, 0xBA, 0xBB, 0xBC, 0xBD, 0xBE, 0xBF
                    .db          0xC0, 0xC1, 0xC2, 0xC3, 0xC4, 0xC5, 0xC6, 0xC7,
0xC8, 0xC9, 0xCA, 0xCB, 0xCC, 0xCD, 0xCE, 0xCF
                    .db          0xE0, 0xE1, 0xE2, 0xE3, 0xE4, 0xE5, 0xE6, 0xE7,
0xE8, 0xE9, 0xEA, 0xEB, 0xEC, 0xED, 0xEE, 0xEF
                    .db          0xFF, 0xFE, 0xFD, 0xFC, 0xFB, 0xFA, 0xF9, 0xF8,
0xF7, 0xF6, 0xF5, 0xF4, 0xF3, 0xF2, 0xF1, 0xF0
                    // please correct end of this table according to the guidelines
given by the teacher
                    .db    0x00,0x00
.EXIT
//-----------------------------------------------------------------------------
```

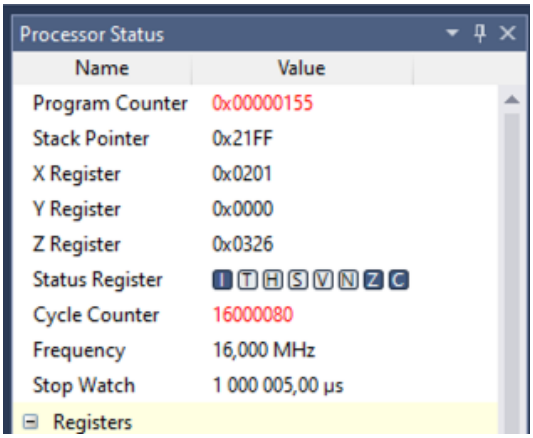
3. Pomiar Czasu

Na potrzeby pomiaru czasu tablicę TAB_ROM ograniczono do 1 elementu, również parametr xlength ograniczono do 1. Ustawiono breakpointy według oznaczeń w kodzie. Pomiary prezentowały się następująco:



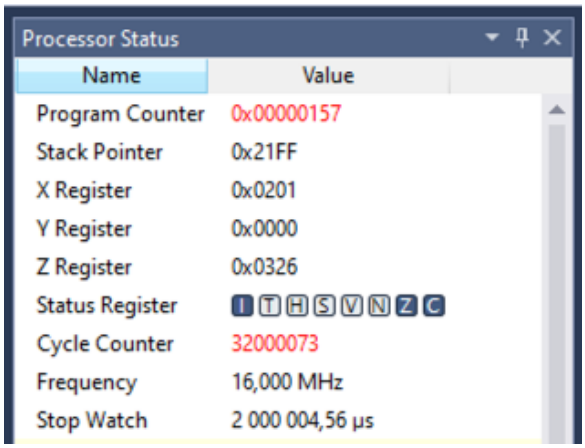


Moment wyświetlenia wyjścia na porcie C: przed i po. Czas trwania:
1 000 004,69 µs – 4,19 µs = 1 000 000,50 µs = 1,0000005 s.



Processor Status

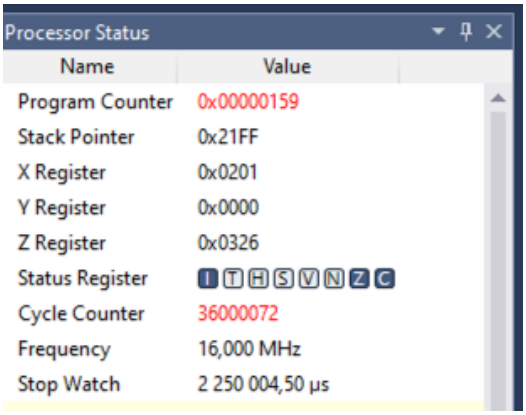
Name	Value
Program Counter	0x00000155
Stack Pointer	0x21FF
X Register	0x0201
Y Register	0x0000
Z Register	0x0326
Status Register	I T H S V N Z C
Cycle Counter	16000080
Frequency	16,000 MHz
Stop Watch	1 000 005,00 µs
Registers	



Processor Status

Name	Value
Program Counter	0x00000157
Stack Pointer	0x21FF
X Register	0x0201
Y Register	0x0000
Z Register	0x0326
Status Register	I T H S V N Z C
Cycle Counter	32000073
Frequency	16,000 MHz
Stop Watch	2 000 004,56 µs

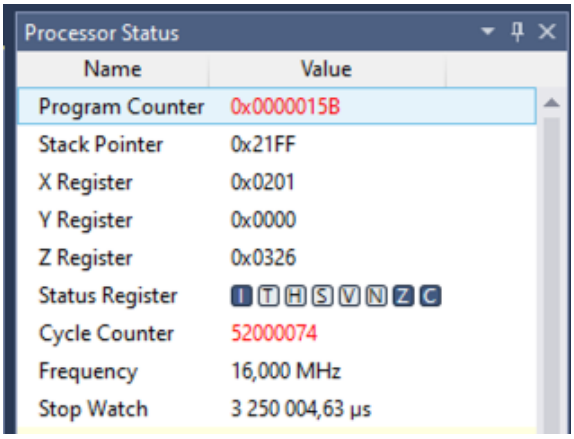
Moment zapalenia diody podłączonej do portu B.7: przed i po. Czas trwania:
2 000 004,56 µs – 1 000 005,00 µs = 999 999.44 µs = 0,999 999 44 s.



Processor Status

Name	Value
Program Counter	0x00000159
Stack Pointer	0x21FF
X Register	0x0201
Y Register	0x0000
Z Register	0x0326
Status Register	I T H S V N Z C
Cycle Counter	36000072
Frequency	16,000 MHz
Stop Watch	2 250 004,50 µs

Moment zgaszenia diody podłączonej do portu B.7: po. Czas trwania:
2 250 004,50 µs – 2 000 004,56 µs = 249 999.94 µs = 249.99994 ms.



Processor Status

Name	Value
Program Counter	0x0000015B
Stack Pointer	0x21FF
X Register	0x0201
Y Register	0x0000
Z Register	0x0326
Status Register	I T H S V N Z C
Cycle Counter	52000074
Frequency	16,000 MHz
Stop Watch	3 250 004,63 µs

Moment ponownego zaświecenia diody podłączonej do portu B.7: po. Czas trwania: 3 250 004,64 µs – 2 250 004,50 µs = 1 000 000.14µs = 1.00000014s.

Podsumowanie użycia pamięci (Flash 808 bajtów, RAM 100 bajtów):

```
"ATmega2560" memory use summary [bytes]:
Segment   Begin    End      Code   Data   Used    Size   Use%
---------------------------------------------------------------
[.cseg] 0x000000 0x000328    566    242    808  262144   0.3%
[.dseg] 0x000200 0x000264      0    100    100    8192   1.2%
[.eseg] 0x000000 0x000000      0      0      0    4096   0.0%
Assembly complete, 0 errors. 0 warnings
```

4.Podsumowanie

Program pomyślnie realizuje algorytm kopiowania danych z pamięci Flash do SRAM z uwzględnieniem filtrowania wartości poniżej **0x80** oraz mechanizmu zapętlania tabeli ROM. Dzięki zastosowaniu instrukcji elpm oraz skoków dalekich jmp wyeliminowano błędy zasięgu skoków względnych oraz zapewniono poprawne adresowanie w rozszerzonej przestrzeni pamięci mikrokontrolera ATmega2560. Wykorzystanie metody odpytywania flagi przepełnienia licznika Timer 1 umożliwiło precyzyjne odliczanie interwałów czasowych bez konieczności stosowania procedur przerwań, co znacząco ułatwiło dokumentację wyników w narzędziu Stopwatch. Pełna inicjalizacja portów od A do L oraz dostosowanie programu do logiki dodatniej wynikającej z pozycji zworki **SV1** zapewniło pełną kompatybilność sprzętową z nakładkąoraz stabilną pracę układu.